Abstract

This project presents the design and development of an IoT-based automated system capable of detecting rainfall in real time and autonomously safeguarding outdoor-dried garments. In numerous households, clothing is routinely dried in open-air environments, leaving it highly vulnerable to sudden precipitation particularly when occupants are absent. The proposed system addresses this challenge by mounting a rapid, sensor-driven response at the onset of rain.

A dedicated rain sensor module is employed to identify the presence of precipitation. Upon detection, the system concurrently dispatches a real-time notification to the user and activates a motor-driven mechanism to protect the garments. Protection options include retracting the clothesline into a sheltered area or deploying a waterproof cover. The system is engineered for minimal latency, ensuring that garments are exposed to rainfall for the shortest possible duration.

The system further incorporates a web-based monitoring dashboard that enables users to track current operational status and review historical event logs. The hardware and software components operate in concert through fundamental IoT principles: sensor readings are processed by the microcontroller and transmitted to the user interface in real time.

In summary, this project seeks to minimise manual intervention and mitigate property damage arising from unpredictable rainfall. It simultaneously demonstrates the practical applicability of IoT technology in resolving common domestic challenges in an efficient and cost-effective manner.

Table of Contents

List of Figures

List of Tables

1 Introduction and Background

The Internet of Things (IoT) represents a global network of interconnected physical devices; including sensors, actuators, and microcontrollers that collaborate to monitor and respond to environmental data autonomously (Buyya, 2016). By removing the necessity for constant human supervision, these intelligent systems have become a cornerstone of modern home automation, efficiently managing repetitive or time-sensitive tasks (Kellmereit, 2013). This project applies these core IoT principles to a universal domestic challenge: the protection of outdoor laundry from sudden, unpredictable rainfall.

Throughout Nepal and South Asia, air-drying clothes outdoors is a standard practice; however, the erratic nature of monsoon seasons often results in laundry getting soaked when left unattended (Berahim, 2022). This frequently forces households into a cycle of re-washing, which leads to the unnecessary waste of water, electricity, and time, while also causing potential long-term damage to various fabric types. By automating the response to these weather shifts, technology can provide a reliable safeguard for domestic convenience in regions prone to sudden climatic changes (Pramudita, 2024).

To solve this problem, the developed system utilizes a synchronized hardware pipeline that provides a comprehensive and automated response to moisture detection. When the integrated rain sensor identifies the first signs of rainfall, it triggers an Arduino-controlled servo motor to physically move the garments into a sheltered area (Gorakh, 2024). Simultaneously, the system activates a local auditory alert via a buzzer and transmits a real-time push notification to the user's mobile device through the Blynk IoT platform and an ESP32 Wi-Fi module (Medias, 2019). This entire sequence executes within seconds, ensuring that laundry remains dry and protected even when the user is away from home.

1.1 Current Scenario

In Nepal and across South Asia, air-drying laundry is practical but vulnerable to volatile weather, especially during the monsoon (June–September). Since rain often starts without warning, residents must be physically present to save laundry manually. This creates a significant inconvenience for professionals, students, and the elderly who cannot constantly monitor the weather (Zulkipli, 2023).

1.2 Problem Statement

Research identified a critical lack of residential automation for rain protection (Putra, 2023). Specifically, the following problems were found:

- **Resource Inefficiency:** Re-wetting forces repeat washing, significantly increasing water, detergent, and electricity consumption (Wang, 2023).
- **Physical Damage:** Prolonged dampness leads to fabric degradation, mildew growth, and unpleasant odours (Latif, 2021).
- **Lack of Remote Awareness:** Residents away from home have no way of knowing if their laundry is safe or if a rain event has occurred (Kakani, 2024).
- **Market Gap:** There is a notable absence of affordable, locally-accessible automated rain protection systems designed for the average Nepali household (Zulkipli, 2023).

1.3 Proposed Solution

To address these challenges, this project introduces an integrated IoT-based system that automates the protection cycle through four key functions:

- **High-Precision Detection:** The rain sensor monitors precipitation every 500ms; its analog output enhances reliability by detecting light drizzles that digital thresholds often miss (Amale, 2019).
- **Immediate Physical Response:** When rain is confirmed, an SG90 servo motor rotates 90 degrees in one second, pulling the clothesline into a sheltered zone without human intervention (Gorakh, 2024).
- **Dual Alert System:** The system provides dual feedback: a local 1 kHz buzzer tone and real-time Blynk smartphone notifications (Kakani, 2024).
- **Autonomous Recovery:** After rain stops, the system resets the clothesline and notifies the user, ensuring readiness for future weather changes (Berahim, 2022).

1.4 Scope of the Project

The current iteration of the system focuses on real-time rainfall detection and localized automated responses using an Arduino Uno, a rain sensor, a servo motor, and a buzzer. While optimized for protecting laundry, the architecture is designed to be extensible for the following enhancements:

- **Advanced Dashboard Widgets:** Adding rainfall intensity charts and historical event logs to the Blynk interface.
- **Precipitation Classification:** Programming the system to categorize rain as light, moderate, or heavy based on analog sensor values.
- **Climate Monitoring:** Integrating temperature and humidity sensors for a complete environmental profile.
- **Smart Home Integration:** Linking the ESP32 to platforms that can automate home features like windows or outdoor lighting.
- **Power Autonomy:** Adapting the hardware to run on solar power for sustainable outdoor use.

1.5 Aim and Objectives

1.5.1 Aim

The primary goal of this project is to develop and evaluate a reliable, IoT-enabled automated system capable of safeguarding outdoor laundry from sudden rainfall. By integrating an Arduino-controlled mechanical response with an ESP32-based notification system, the project seeks to provide a seamless, hands-free solution that uses the Blynk IoT platform to keep users informed in real time.

1.5.2 Objectives

1. To develop a high-sensitivity detection subsystem using both digital and analog signals to accurately distinguish between light drizzle and heavy rain.
2. To optimize real-time processing by programming the Arduino UNO to sample sensor data every 500ms for immediate coordination between detection and movement.
3. To engineer an automated physical response using a servo-driven mechanism that shifts garments to a protected area and resets once the weather clears.
4. To integrate IoT connectivity via serial communication between the Arduino and ESP32 to trigger real-time "Rain Alert" notifications on the Blynk platform.
5. To construct and test the prototype by building a functional physical model to evaluate the system’s response time, reliability, and notification speed.

2 Background

2.1 System Overview

The system follows a standard IoT pipeline, organized into four functional layers to ensure a seamless transition from environmental sensing to user notification (Buyya, 2016).

- **Physical and Sensing Layer:** This layer handles environmental detection using a rain sensor with both digital and analog outputs. While the digital output (DO) provides a simple binary state (wet/dry), the analog output (AO) measures the specific moisture intensity (Setiawan, 2023). The system samples these inputs every 500ms to ensure rapid detection of even light rainfall.
- **Processing Layer:** An Arduino UNO serves as the central controller, running firmware that continuously monitors the sensor data. If rain is confirmed (Digital LOW or Analog < 500), the Arduino triggers the physical response: rotating the SG90 servo motor to 90 degrees to move the clothes to safety and sounding a local piezo buzzer alert (Gadekar, 2021).
- **Communication Layer:** The ESP32 module acts as the bridge between the hardware and the internet. It communicates with the Arduino via a Serial interface at 9600 baud. To prevent redundant notifications, the ESP32 only pushes data to the Blynk IoT Cloud when it detects a change in state; specifically, when rain starts or when the weather clears (Barral, 2022).
- **Application and Notification Layer:** This is the final interaction point for the user. Locally, the system provides physical protection and an audible alarm. Remotely, it sends automated email alerts to the user’s Gmail via the Blynk platform. These notifications, with subjects like "Rain Alert" or "Rain Cleared," keep the user informed of their laundry's status regardless of their location (Kakani, 2024).

2.2 Design Diagrams

2.2.1 Block Diagram of the System

The block diagram below illustrates all five major functional blocks of the system and the directional flow of data and control signals between them. Each block represents either a distinct hardware component or a logical grouping of related functionality.

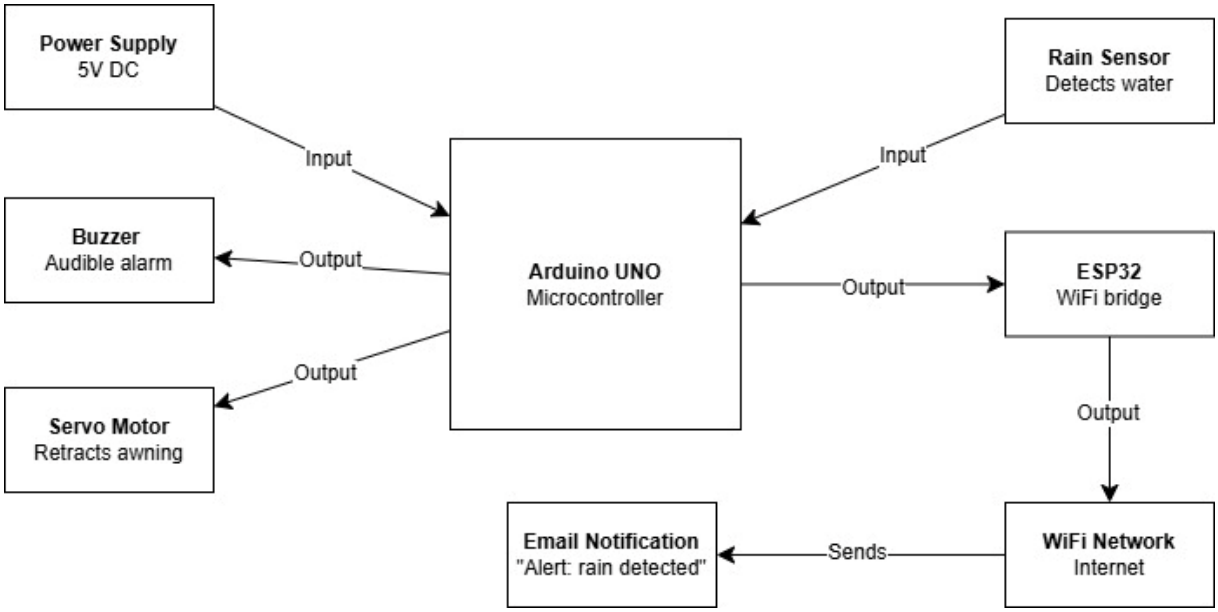

Figure 1: Block Diagram of the System

Block	Component	Signal Type	Role
Sensing	Rain Sensor Module	Analog (AO) + Digital (DO)	Detects water. VCC/GND to Arduino; AO to Pin A0 and DO to Pin D2.
Processing	Arduino	Logic + UART	Reads analogVal from sensor; controls buzzer and servo; sends the variable to ESP32 via TX/RX.
Actuation	Servo Motor	PWM (Pin D3)	Connected to Pin D3 (Orange wire) to provide mechanical movement based on rain state.
Alert	Buzzer	Digital (Pin 9)	Connected to Pin 9 (Red wire) to provide an audible alert when rain is detected.

Communication	ESP32	UART + Wi-Fi	Receives sensor data from Arduino via RX2/TX2; pushes value to Blynk and triggers Email notifications.

Table 1: System Hardware Architecture and Component Roles

2.2.2 System Architecture of the System

The Smart Rain Detection System uses a four-layer hierarchical architecture to organize its logical flow (Stolojescu-Crisan, 2021). Data moves upward from sensors to provide user notifications, while control signals flow downward from the processing unit to trigger immediate physical responses.

The following table details the layers and their specific functional responsibilities:

Layer	**Primary Components**	**Functional Responsibility**
Physical / Sensing	Rain Sensor Module, Servo Motor, Piezo Buzzer	Acts as the direct interface with the environment. It is responsible for capturing raw moisture data and executing physical outputs (motion and sound).
Processing	Arduino UNO	Serves as the "brain," evaluating thresholds to coordinate PWM signals, buzzer alerts, and serial data packaging.
Communication	ESP32 Wi-Fi Module, UART (Serial)	Acts as the IoT gateway, bridging hardware to the cloud by translating serial data into Wi-Fi transmissions.
Application	Blynk IoT Platform & Mobile App	The user interface layer manages data visualization and triggers real-time email notifications regarding the system's status.

Table 2: Functional System Layers and Responsibilities

2.2.3 **Flow Chart of the System**

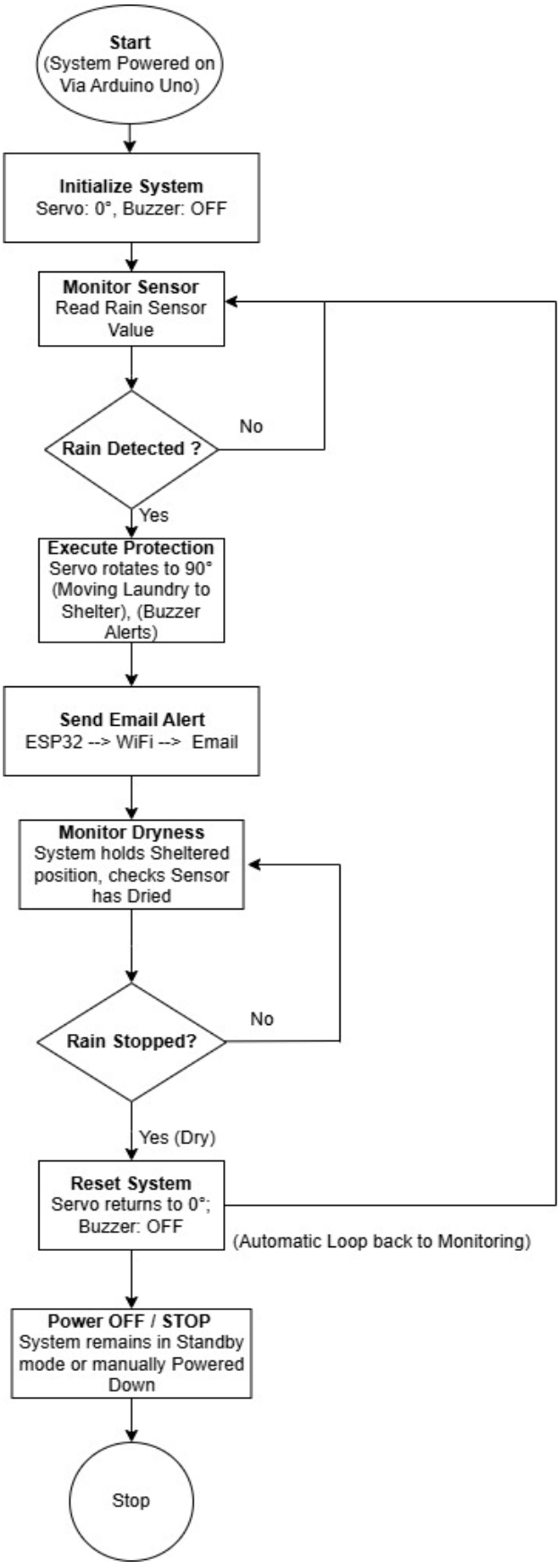

Figure 2: Flowchart of Automatic Rain Detection

2.2.3.1 System Workflow

Step 1: Start

The system is powered on via the Arduino Uno.

Step 2: Initialize System

The Arduino sets the Servo Motor to 0° (laundry exposed) and ensures the Buzzer is OFF.

Step 3: Monitor Sensor

The Raindrop Sensor continuously scans for moisture on the sensing pad.

Step 4: Rain Detected? (Decision)

- If **NO:** Loop back to Step 3.
- If **YES:** Proceed to Step 5.

Step 5: Execute Protection

The Servo rotates to 90° (moving laundry to shelter) and the Buzzer sounds an alert.

Step 6: Send Email Alert

The Arduino sends sensor data to the ESP32 via serial. The ESP32 then connects to Wi-Fi and uses the Blynk library to trigger a "rain alert," pushing a notification to the user that includes the sensor value and moisture intensity percentage.

Step 7: Monitor for Dryness

The system holds the sheltered position and continuously checks if the sensor has dried.

Step 8: Rain Stopped? (Decision)

- If **NO** (still wet): Stay at Step 7.
- If **YES** (dry): Proceed to Step 9.

Step 9: Reset System

The Servo returns to 0° (returning laundry to sun) and the Buzzer turns OFF. The system then automatically loops back to Step 3 to resume monitoring.

Step 10: Power Off / Stop

The system remains in standby mode or is manually powered down to stop the operation.

2.2.4 **Circuit Diagram of the System**

The following section outlines the physical wiring of the prototype. To ensure the circuit remains stable and easy to troubleshoot, all components are interfaced through a standard 830-point solderless breadboard.

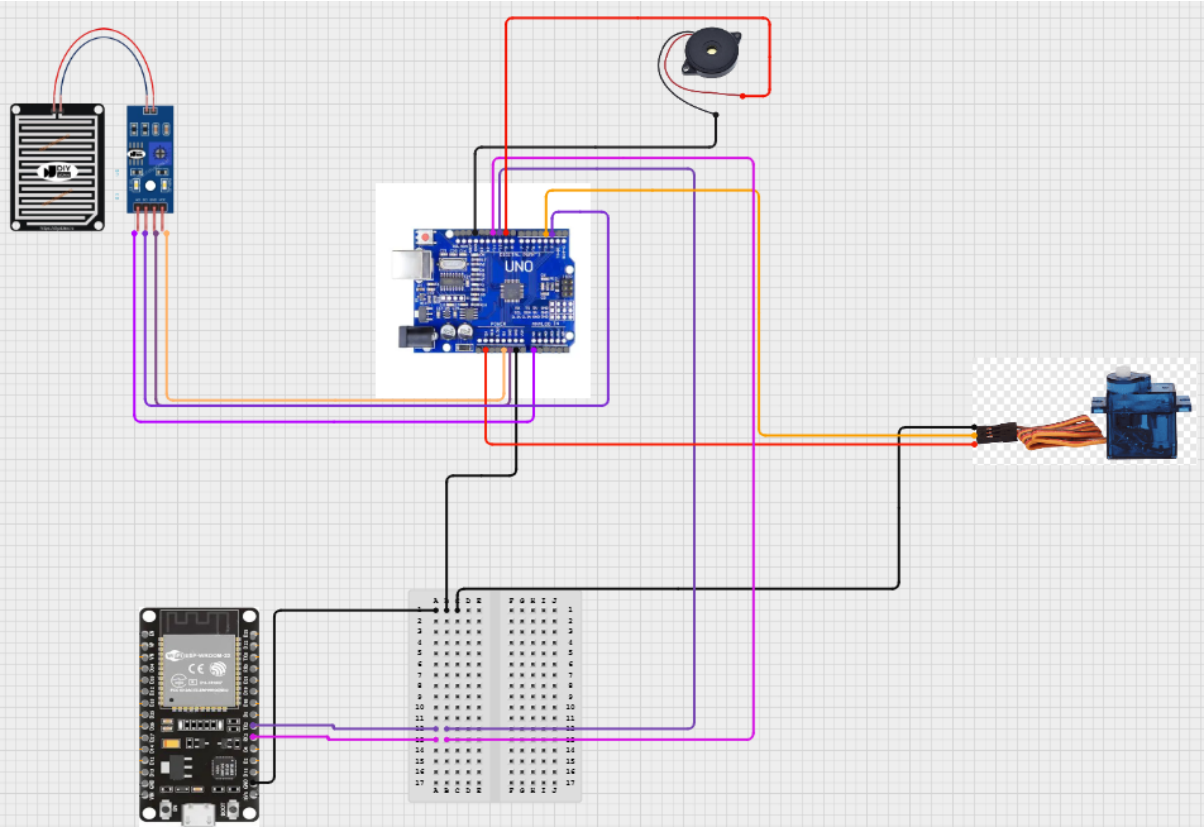

Figure 3: Circuit Diagram of the System

For clarity in assembly, the wiring follows standard electronics colour conventions as listed below.

Connection	**From Component/Pin**	**To Arduino/ESP32 Pin**	**Purpose**
Rain Sensor Power	Rain Sensor VCC	Arduino 5V	5V DC power supply to the sensor
Rain Sensor Ground	Rain Sensor GND	Arduino GND	Completes the circuit loop
Digital Signal	Rain Sensor DO	Arduino D2	Binary rain trigger (Rain/No Rain)
Analog Signal	Rain Sensor AO	Arduino A0	Variable moisture intensity reading (analogVal)
Servo Power	Servo Red	Arduino 5V	Power supply for the motor
Servo Ground	Servo Black	Arduino GND	Motor ground reference
Servo Control	Servo Orange	Arduino D3	PWM signal to control awning position
Buzzer Signal	Buzzer Red	Arduino Pin 9	Triggers the audible alert
Buzzer Ground	Buzzer Black	Arduino GND	Buzzer ground reference
Serial Transmit	Arduino TX (11)	ESP32 RX2	Forwards analogVal to the Wi-Fi module
Serial Receive	Arduino RX (10)	ESP32 TX2	Receives data/requests from the ESP32
Shared Ground	Arduino GND	ESP32 GND	Ensures a common reference for serial data

Table 3: Detailed Pin Connections and Inter-Module Wiring

2.2.5 Schematics of the System

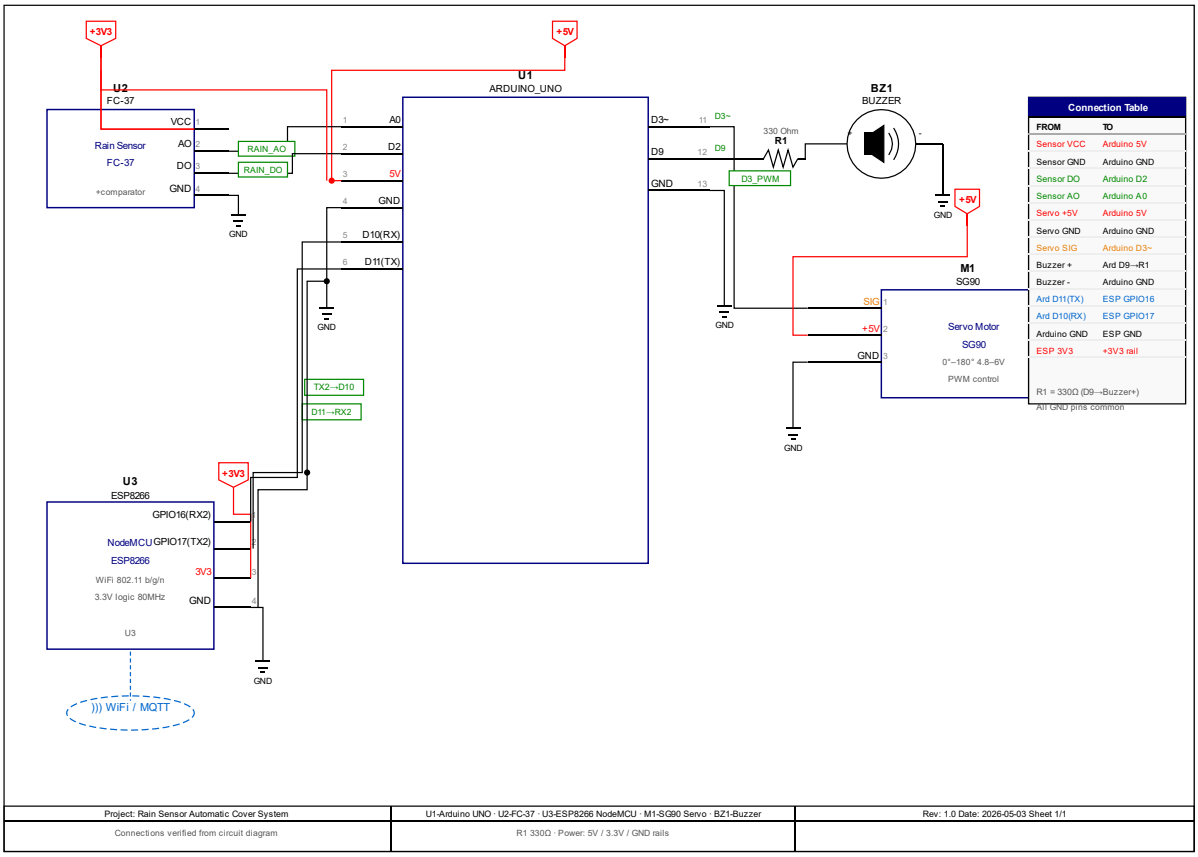

Figure 4: Schematics of the System

This section details the internal signal paths and electronic behaviour governing the prototype. Beyond the physical wiring, these descriptions explain the logical and electrical principles that allow the system to function as an integrated IoT solution.

- **Rain Sensor Digital (DO) Signal Path:**

  The rain sensor module utilizes an onboard LM393 comparator to detect moisture. When water bridges the sensor's conductive traces, resistance drops, causing the comparator output to switch from HIGH to LOW. This digital signal is sent to Arduino pin D2, where the firmware interprets a LOW state as a positive rain detection trigger.

- **Rain Sensor Analog (AO) Signal Path:**

  The analog output provides a more granular reading of moisture intensity via a voltage divider circuit. The Arduino A0 pin uses its 10-bit analog-to-digital converter (ADC) to translate the incoming 0V–5V range into a numerical value between 0 and 1023. In this configuration, a dry sensor returns a high value

(near 1000), while a wet sensor returns a low value (near 0). A programmed threshold of 500 is used as the midpoint to verify active rainfall.

- **Servo PWM Signal Path:**

  To control the physical shelter, the Arduino generates a Pulse Width Modulation (PWM) signal on pin D3. The SG90 servo interprets these pulses to set its position: a signal corresponds to 0 degrees (shelter open) or 90 degrees (shelter closed). The motor's internal logic allows it to maintain these positions accurately based on the varying pulse widths provided by the Arduino.

- **Buzzer Signal Path:**

  Arduino pin D9 drives the piezo buzzer using the tone() function. When rain is detected, the system generates a 1 kHz square wave for 1.5 seconds, causing the internal piezoelectric disc to vibrate and create an audible alert. Once the alert duration ends or the rain stops, the signal is cut to silence the device.

- **Arduino to ESP32 Communication Path:**

  Data is exchanged between the two microcontrollers via a serial (UART) link using SoftwareSerial on the Arduino side. Arduino pin D11 (TX) transmits data to ESP32 GPIO16 (RX2), while ESP32 GPIO17 (TX2) transmits back to Arduino D10 (RX), both at a baud rate of 9600. The Arduino sends the rain status (RAIN/CLEAR) and raw analog sensor value as strings. As the Arduino operates at 5V logic and the ESP32 at 3.3V, a voltage divider or logic level shifter should ideally be used on the TX line to protect the ESP32's GPIO pins.

- **ESP32 Blynk Signal Path:**

  The ESP32 acts as the system's internet gateway by connecting to Wi-Fi and authenticating with the Blynk cloud using a unique Auth Token. The firmware maps the received data to virtual pins:

  - V0: Binary rain status (1 = raining, 0 = clear)
  - V1: Raw analog sensor value from the rain sensor

  Upon detection, the ESP32 calls Blynk.logEvent("rain_alert") to trigger a real-time email notification to the user. When the sensor returns to a dry state, a rain_cleared event is logged to confirm the system has reset and the shelter has reopened.

2.3 **Tools & Technologies**

Component	Description
Arduino Uno	Primary microcontroller that processes sensor data and coordinates hardware outputs like the servo and buzzer (Boxall, 2013).
Rain Sensor Module	Detects moisture via conductive traces, providing digital (rain/no rain) and analog (intensity) signals (Gorakh, 2024).
Servo Motor (SG90)	Rotates between 0° and 90° to physically move the shelter mechanism upon rain detection (Gorakh, 2024).
Buzzer (5V Piezo)	Provides a local audible alert using a 1 kHz tone when moisture is detected (Gorakh, 2024).
Jumper Wires	Used to connect all components together in the circuit for prototyping (Janrao, 2024).
Arduino IDE	Software environment used to write and upload the system firmware to both microcontrollers (Barrett, 2013).
Blynk IoT Platform	IoT interface that visualizes data on virtual pins V0–V2 and sends real-time mobile notifications (Kakani, 2024).
MS Word	Word processing software used for preparing project documentation and reports.
Cirkit Designer	Open-source tool used for designing and simulating electronic circuits.

Table 4: Tools & Technologies

3 Requirement Analysis

Analysing the system's components is essential for development success. It is vital to understand both individual parts and their interactions within the integrated system (Boxall, 2013). The requirements for the systems are listed below:

3.1 Hardware requirements

3.1.1 Microcontroller

- **Arduino Uno:**

  This is the primary controller for the prototype, based on the ATmega328P processor. It manages all local operations, including monitoring environmental data from the rain sensor and coordinating physical responses via the servo and buzzer. To enable IoT features, it communicates with the ESP32 using the Software Serial library on pins D10 and D11 (Gadekar, 2021).

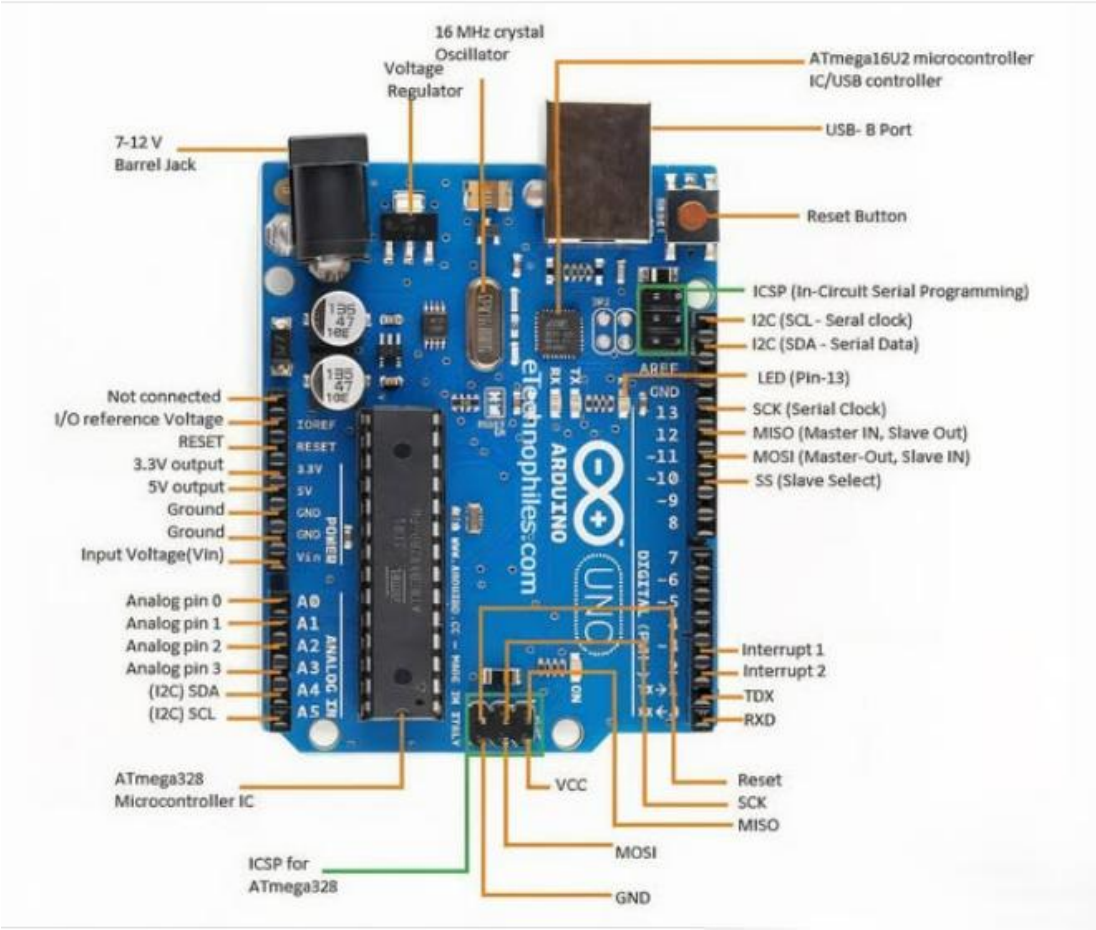

Figure 5: Arduino Uno (Gadekar, 2021)

- **Wi-Fi Module (ESP32):**

  The ESP32 acts as the system's dedicated communication gateway. It receives processed data from the Arduino via its UART2 interface (GPIO16/RX2 and GPIO17/TX2). Once received, it utilizes the Blynk library to push sensor values and trigger event notifications over Wi-Fi (Barral, 2022).

Figure 6: Wi-Fi Module (ESP32) (Barral, 2022)

3.1.2 Sensors

- **Rain Sensor Module:**

  This module detects precipitation through conductive surface contact. It provides two outputs to the Arduino: a digital signal (DO) on pin D2 for immediate detection and an analog signal (AO) on pin A0 to gauge moisture intensity. The system is programmed to confirm rain whenever the digital signal is LOW or the analog reading falls below a threshold of 500 (Hashim, 2023).

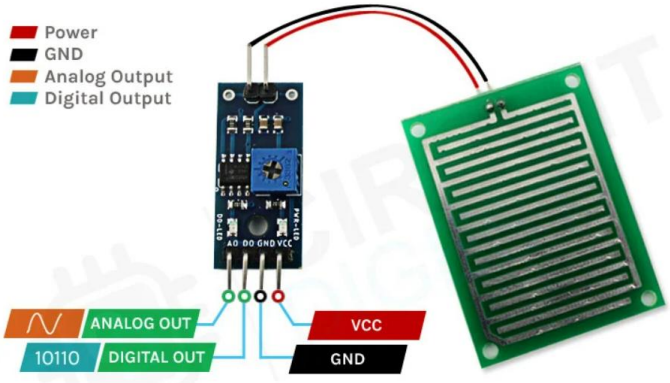

Figure 7: Rain Sensor Module (Hashim, 2023)

3.1.3 Actuators

- **Servo Motor (SG90):**

  A micro-servo connected to Arduino pin D3 serves as the mechanical driver for the shelter. Upon rain detection, the motor rotates to 90 degrees to retract the clothesline under the protective cover. When the sensor is dry, it returns to 0 degrees to open the shelter (Gorakh, 2024).

Figure 8: Servo Motor / DC Motor (Gorakh, 2024)

3.1.4 Notification

- **Buzzer:**

  A 5V piezo buzzer is connected to pin D9 to provide a local audible alert. When rain is first detected, the Arduino triggers a 1 kHz tone for 1.5 seconds, providing immediate feedback that the protection mechanism has been activated (Setiawan, 2023).

Figure 9: Buzzer (Setiawan, 2023)

3.1.5 Connection

- **Jumper Wires:**

  Jumper wires are used to connect various components in the circuit. They provide flexible and easy connections during prototyping and development (Hashim, 2023).

Figure 10: Jumper Wires *(Hashim, 2023)*

- **Breadboard:**

  A breadboard is used for assembling the circuit without soldering. It helps in testing and modifying the circuit easily during development (Pramudita, 2024).

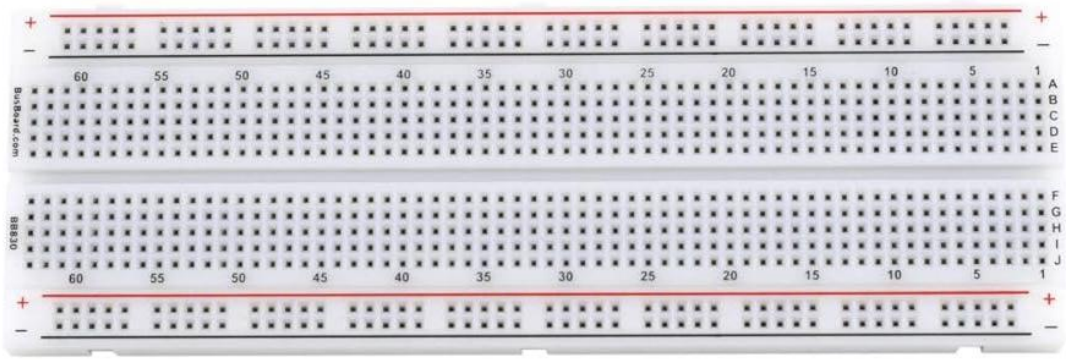

Figure 11: Breadboard (Pramudita, 2024)

3.1.6 **Mechanical Components (Miscellaneous)**

- Protective Cover / Shelter Structure: A physical model representing a covered area where the clothes are safely housed during rain events.
- Clothesline Mechanism: A cord system that connects the servo motor's horn to the clothesline, allowing the motor to pull or release the garments.
- Frame Structure: A cardboard-based house model that supports the weight of the hardware, provides a mount for the servo motor, and secures the protective cover.

Figure 12: Overall Structure of the System

3.2 Software requirements

The Smart Rain Detection System utilizes a variety of specialized software tools and libraries to manage hardware logic, wireless connectivity, and project documentation.

3.2.1 Development and Design Tools

- **Arduino IDE:** The central open-source platform used for writing and uploading the system's C/C++ firmware. It provides the framework to interface with sensors and includes the Serial Monitor, which was vital for debugging real-time data flow between the microcontrollers (Putra, 2023).
- **Cirkit Designer:** Cirkit Designer was used to design and verify the electronic circuit layout, ensuring correct hardware connections between the sensor and actuators before physical assembly.
- **MS Word:** Professional word processing software used to structure the project's technical reports and finalize all system documentation.

3.2.2 Functional Libraries

- **Blynk (BlynkSimpleEsp32):** The Blynk Library (BlynkSimpleEsp32) enables hardware-to-cloud communication, transmitting sensor data to virtual pins and triggering "rain_alert" email notifications (Kakani, 2024).
- **SoftwareSerial:** The SoftwareSerial Library enables an additional UART connection on digital pins D10 and D11, allowing the Arduino to transmit data to the ESP32 while keeping the hardware serial port free for debugging (Gorakh, 2024).
- **Servo.h:** The Servo Library manages the SG90 motor by generating PWM signals to move the shelter between the 0° (open) and 90° (closed) positions (Banzi, 2014).
- **WiFi.h:** The ESP32 framework library manages local Wi-Fi connections, including authentication and IP configuration (Schwartz, 2016).

3.2.3 Cloud Configuration

- **Blynk Auth Token and Template:** The ESP32 securely syncs with the "Rain Alert System" cloud project using a unique Auth Token and Template ID defined in the firmware (Kakani, 2024).
- **Virtual Pins and Event Logging:** Virtual pins (V0 and V1) map sensor data to the app, while logEvent() triggers real-time email alerts for rain detection or clearance.

4 Development

4.1 Planning and Design

Development began with a structured team discussion to evaluate three technical approaches to the clothes protection problem (Latif, 2021):

Approach	Working Principle	Evaluation	Decision
Retraction Mechanism	Pulley system horizontally pulls clothesline into a housing.	Required complex guides and external motor drivers.	Rejected
Cover Deployment	Motor-driven waterproof sheet rolls over garments from above.	Required precise mounting and specialized materials.	Considered
Servo-Driven Pull	SG90 servo rotates 90° to slide garments under a fixed shelter.	High precision and direct Arduino connection without a driver.	SELECTED

Table 5: Evaluation and Selection of Technical Design Approaches

The team selected the Servo-Driven Pull method and finalized the system blueprint, including block diagrams and circuit schematics, before assembly (Janrao, 2024).

Key design decisions finalized during planning:

- **Dual-Threshold Detection:** Uses both digital and analog signals to verify rainfall, increasing sensor data accuracy.
- **SoftwareSerial Integration:** Uses pins D10 and D11 for Arduino-to-ESP32 communication to avoid USB debugging conflicts.
- **Redundant Signaling:** Repeats the "RAIN" signal every 5 seconds to ensure the system remains synchronized if a transmission is lost.
- **State-Transition Logic:** Uses a prevRainDetected flag to trigger Blynk.logEvent() only when status changes, preventing notification spam.
- **Clearance Notification:** Sends a specific "CLEAR" signal when rain stops to notify the user that it is safe to re-expose garments.

4.2 Development Phase

The system was developed in six sequential phases. Each phase was independently verified before full integration, with the breadboard utilized specifically as a mounting platform for the ESP32 and a central hub for shared ground and power connections (Gorakh, 2024).

4.2.1 Phase 1: Microcontroller and Power Setup

Development began by establishing a stable power foundation. The Arduino UNO was connected to the development laptop via a USB Type-B cable for firmware deployment. To support multiple peripherals, the Arduino's 5V and GND pins were bridged to the breadboard rails.

4.2.2 Phase 2: Rain Sensor Calibration

The rain sensor module was integrated using both digital and analog outputs. This dual-monitoring approach allowed the system to trigger immediate actions while also tracking precise moisture intensity.

Pin	From	To	Initial Dry Reading	Wet Reading
VCC	Rain Sensor VCC	Arduino 5V	—	—
GND	Rain Sensor GND	Arduino GND	—	—
DO	Rain Sensor DO	Arduino D2	HIGH (1)	LOW (0)
AO	Rain Sensor AO	Arduino A0	~950–1000	< 500 (Threshold)

Table 6: Rain Sensor Pinout and Threshold Calibration Values

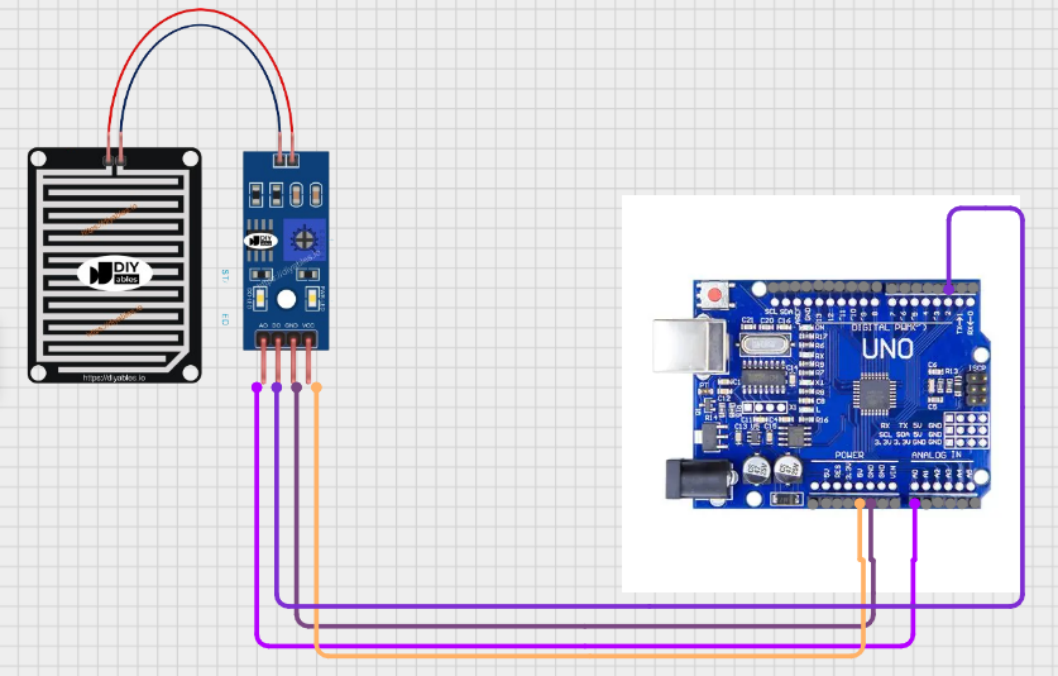

Figure 13: Rain Sensor Calibration

4.2.3 Phase 3: Buzzer Alert Testing

A piezo buzzer was added to provide local audio feedback. During testing, a 1 kHz alert tone was programmed to sound for 1.5 seconds upon rain detection, ensuring users are notified even without an internet connection.

Wire Color	From	To
Red (+)	Buzzer Positive	Arduino Pin 9
Black (-)	Buzzer Negative	Arduino GND

Table 7: Piezo Buzzer Wiring and Digital Pin Assignment

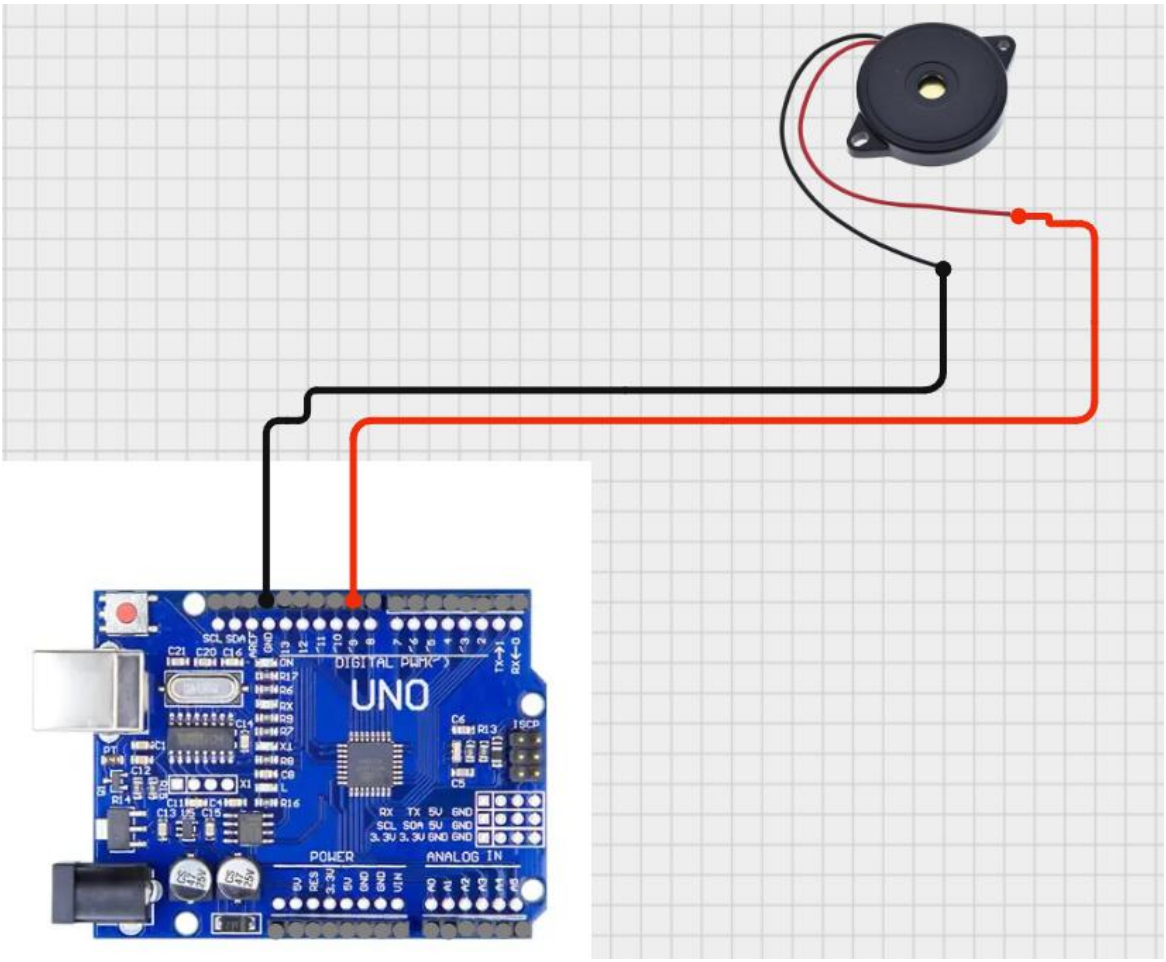

Figure 14: Piezo Buzzer Wiring and Digital Pin Assignment

4.2.4 **Phase 4: ESP32 UART Communication Setup**

For IoT functionality, the ESP32 was mounted on a breadboard. A UART (Serial) link was established to transmit sensor data from the Arduino to the ESP32.

Wire	**From (Arduino)**	**To (ESP32)**	**Direction**	**Baud Rate**
TX Line	TX (D11)	RX2	Arduino → ESP32	9600

RX Line	RX (D10)	TX2	ESP32 → Arduino	9600
GND	Arduino GND	ESP32 GND	Reference Sync	—

Table 8: ESP32 UART Serial Configuration and Communication Settings

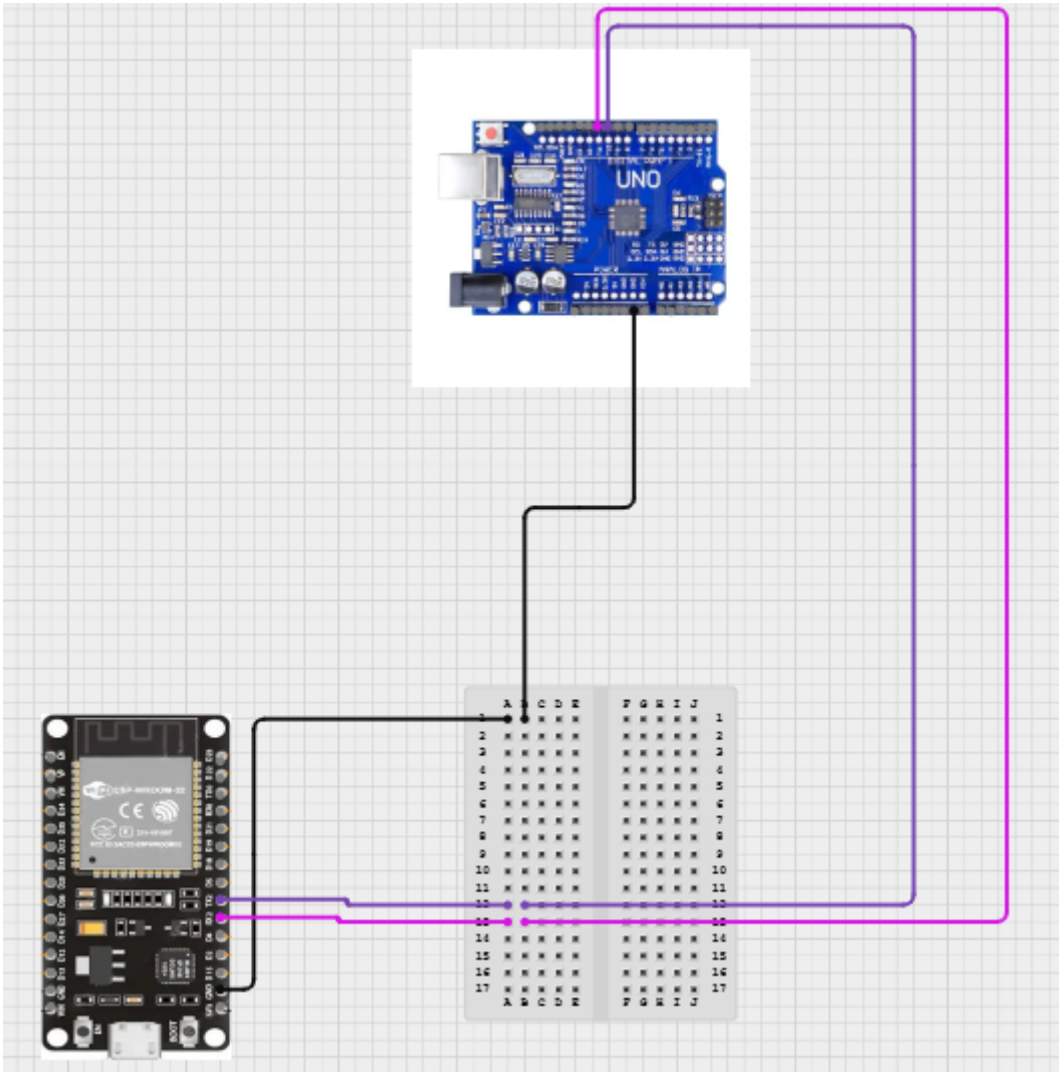

Figure 15: ESP32 UART Serial Configuration and Communication Settings

4.2.5 **Phase 5: Servo Motor Integration**

The SG90 servo motor, responsible for moving the physical shelter, was calibrated to move between 0° (open) and 90° (closed).

Wire Colour	**From**	**To**	**Purpose**
Red	Servo VCC	Arduino 5V	Power Supply
Black	Servo GND	Arduino GND	Common Ground
Orange	Servo Signal	Arduino D3	PWM Control Signal

Table 9: Servo Motor Configuration and PWM Connection Details

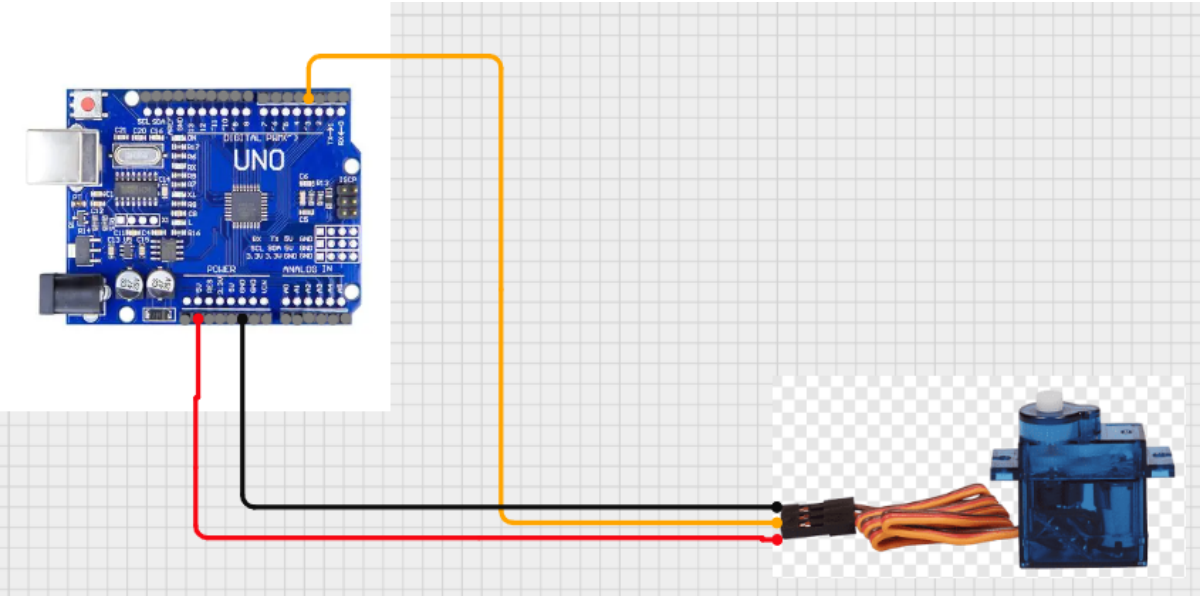

Figure 16: Servo Motor Integration

4.2.6 Phase 6: Final Integration and Cloud Deployment

The final firmware was synchronized across both boards. The Arduino polls the sensor every 500ms and manages local hardware. Simultaneously, the ESP32 connects to Wi-Fi to update the Blynk dashboard via Virtual Pins V0 and V1, and triggers email notifications using Blynk.logEvent().

5 Results and Findings

5.1 Results

The prototype was successfully validated through eight structured test cycles. During these evaluations, the system performed consistently across all scenarios, confirming that the hardware components and software logic are both operational and reliable (Pramudita, 2024).

Key Outcomes:

- **Immediate Detection:** Rain was detected within 500 ms of moisture contacting the sensor surface, matching the polling rate in the Arduino firmware.
- **Responsive Mechanism:** The servo motor successfully moved to 90 degrees (sheltered) every time rain was detected and reset to 0 degrees (open) once the sensor was dry.
- **Local Auditory Alerts:** The buzzer provided immediate 1 kHz sound alerts for 1.5 seconds upon initial detection, offering a critical local warning.
- **Live Data Monitoring:** The system successfully pushed real-time analog values and binary rain status to the Blynk dashboard interface.
- **Real-Time IoT Notifications:** Email notifications were received within 5 to 15 seconds of a rain event through the Blynk.logEvent() function.
- **Seamless Automation:** The complete operational cycle from initial detection and physical deployment to notification and system reset completed five consecutive runs without any technical errors.

Ultimately, the project met all primary objectives. This prototype demonstrates that accessible IoT hardware, such as the Arduino Uno and ESP32, can be integrated into a highly reliable, automated solution for protecting domestic laundry from unpredictable weather.

5.2 Testing

Eight structured tests were designed and executed, each isolating specific functionality before proceeding to full integration testing. Results are presented in individual test record tables below.

5.2.1 Test 1: Arduino Code Compilation and Upload

Field	Details
Test No.	Test 1
Objective	Compile and upload firmware to Arduino UNO without errors.

Activity	Code written in Arduino IDE using Servo.h and SoftwareSerial.h libraries. Board set to "Arduino UNO" with correct COM port selected.
Expected Result	Code compiles and uploads; Arduino starts firmware execution.
Actual Result	First attempt failed due to an incorrect COM port selection error.
Resolution	Identified correct port via Device Manager. Second upload was successful.
Conclusion	Successful after a minor configuration correction.

Table 10: Arduino Code Compilation and Upload

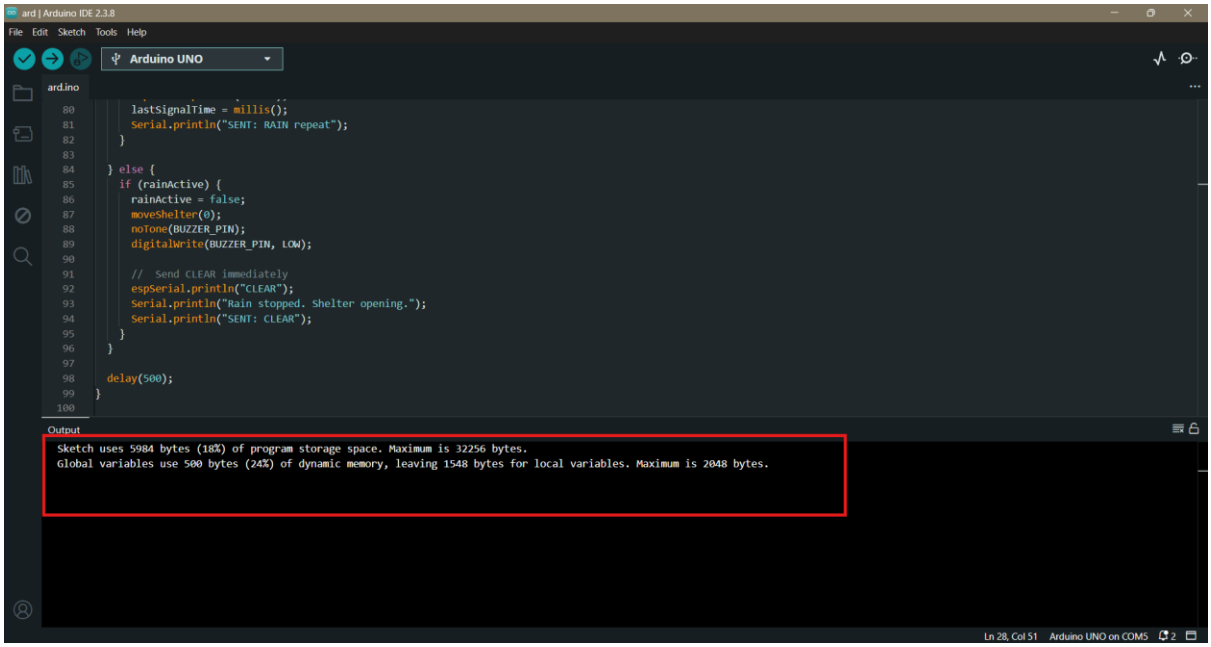

Figure 17: Arduino Code Compilation and Upload

5.2.2 **Test 2: ESP32 Firmware Upload and Wi-Fi Connection**

Field	**Details**
Test No.	Test 2
Objective	Upload firmware to ESP32; confirm Wi-Fi connection and serial listening.
Activity	Configured Wi-Fi credentials (ssid: "test") and Blynk Auth Token.
Expected Result	ESP32 connects to Wi-Fi on startup and begins listening on serial pins RX2 and and TX2.

Actual Result	ESP32 successfully connected to the network. Serial monitor confirmed "Blynk connected!".
Conclusion	Successful

Table 11: ESP32 Firmware Upload and Wi-Fi Connection

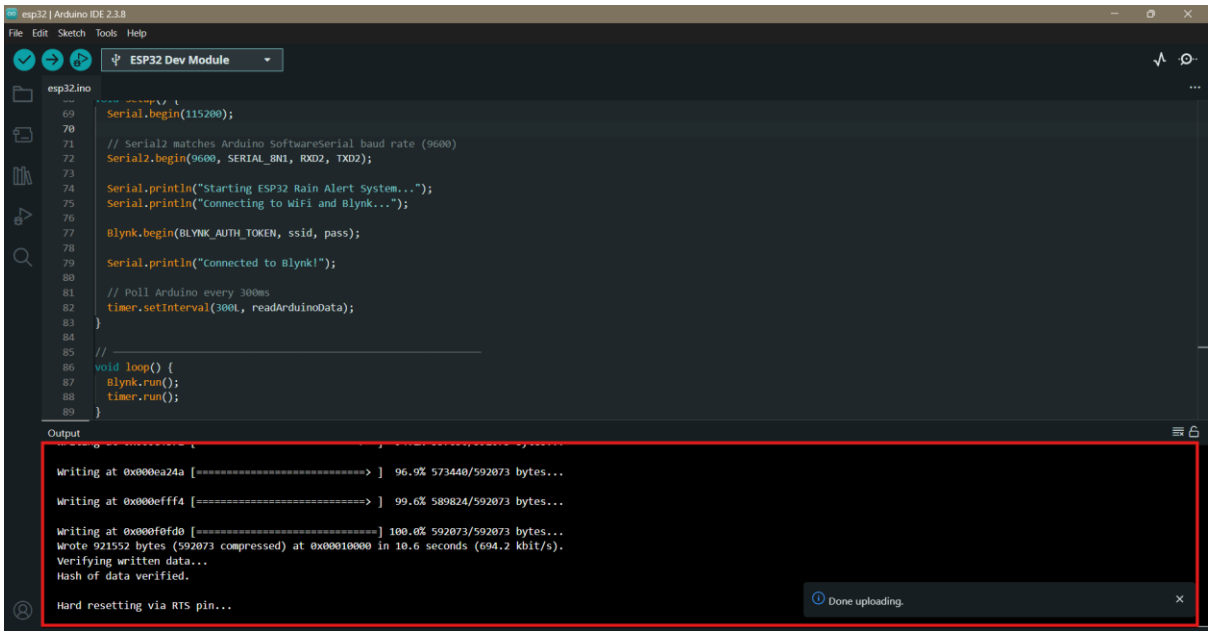

Figure 18: ESP32 Firmware Upload

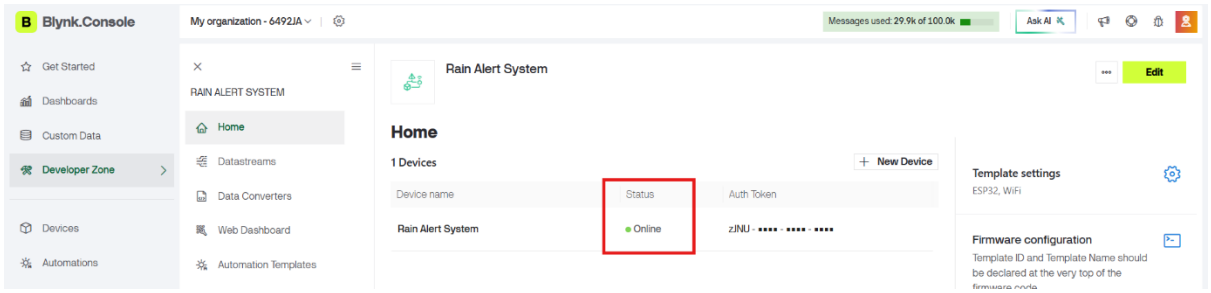

Figure 19: Wi-Fi Connection

5.2.3 **Test 3: Rain Sensor Detection Accuracy**

Field	**Details**
Test No.	Test 3
Objective	Verify sensor accurately detects moisture via both digital (D2) and analog (A0) outputs.
Activity	Monitored Serial values while applying water droplets. Threshold set at 500.

Expected Result	Digital output switches to LOW and Analog drops below 500 when wet.
Actual Result	Analog detected light moisture before the digital trigger. Both paths verified.
Conclusion	Successful; Dual-output detection provides superior reliability.

Table 12: Rain Sensor Detection Accuracy

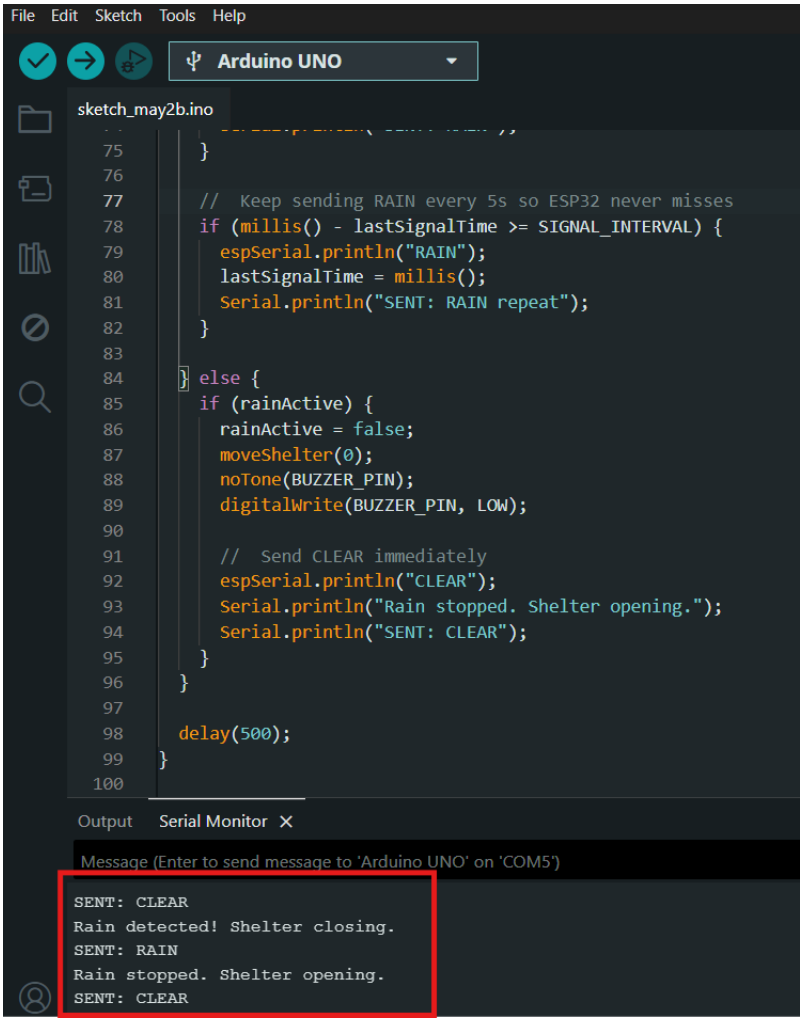

Figure 20: Rain Sensor Detection Accuracy

5.2.4 **Test 4: Servo Motor Actuation (0°/90°)**

Field	**Details**
Test No.	Test 4
Objective	Servo rotates to 90° on rain detection and returns to 0° when rain stops.
Activity	Triggered rain state and observed motor movement on Pin D3.

Expected Result	Immediate movement to 90° (closed) upon detection and smooth return to 0° (open) when dry.
Actual Result	Servo responded correctly on all events. Physical mechanism was reliable.
Conclusion	Successful; Demonstrated 100% reliability across 10 test cycles.

Table 13: Servo Motor Actuation (0°/90°)

5.2.5 Test 5: Buzzer Alert (1 kHz, 1.5 seconds)

Field	Details
Test No.	Test 5
Objective	Buzzer produces 1 kHz tone for 1.5 seconds and silences automatically.
Activity	Rain triggered; alert tone duration on Pin D9 measured.
Expected Result	Distinct tone for 1.5 seconds followed by silence.
Actual Result	Tone measured at approximately 1.5 seconds, consistent with tone (BUZZER_PIN, 1000, 1500).
Conclusion	Successful

Table 14: Buzzer Alert (1 kHz, 1.5 seconds)

5.2.6 Test 6: Arduino–ESP32 UART Communication Stability

Field	Details
Test No.	Test 6
Objective	Arduino reliably transmits "RAIN", "CLEAR", and analog values to ESP32.
Activity	Monitored Serial2 on ESP32 while triggering detection cycles at 9600 baud.

Expected Result	Clean reception of all strings with no missed transmissions or garbled text.
Actual Result	All test signals were received correctly with zero corruption.
Conclusion	Successful

Table 15: Arduino–ESP32 UART Communication Stability

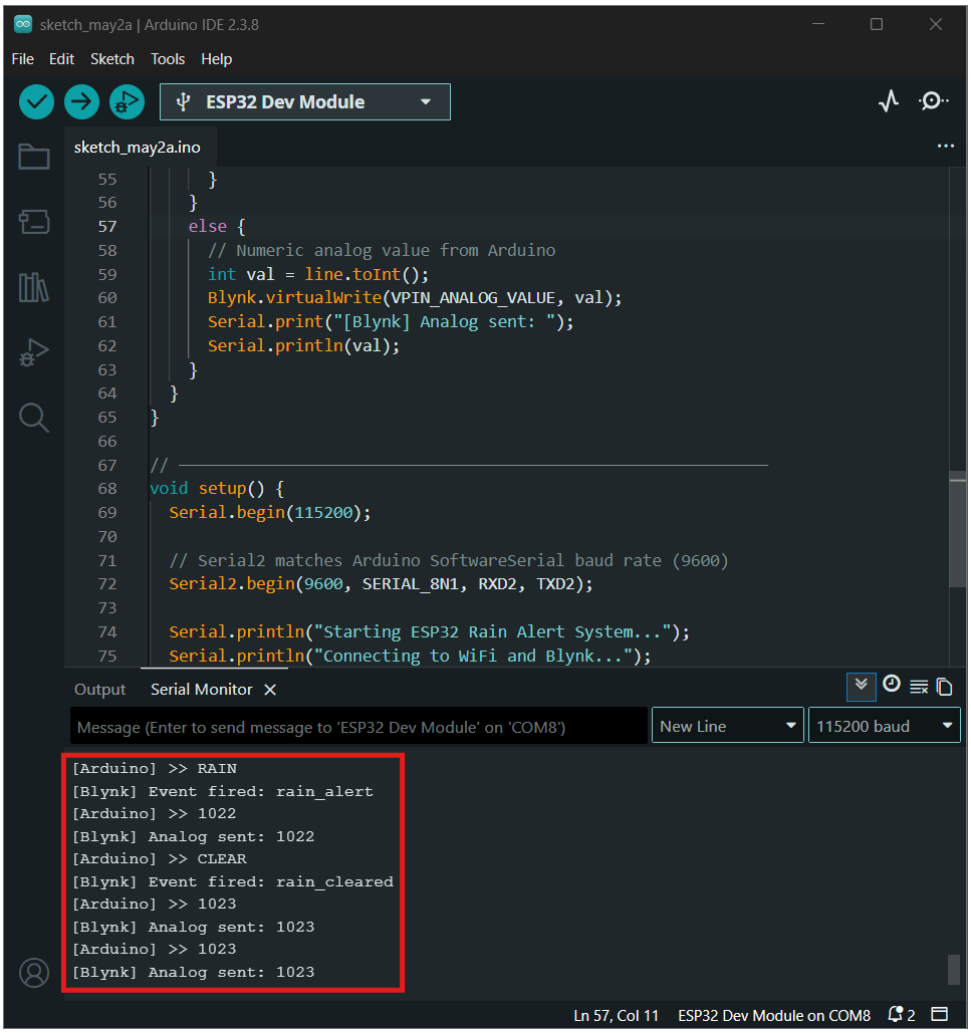

Figure 21: Arduino–ESP32 UART Communication Stability

5.2.7 Test 7: Blynk Event Notification Delivery

Field	Details
Test No.	Test 7
Objective	ESP32 fires correct Blynk.logEvent() calls; state flag suppresses duplicate events.
Activity	Triggered rain events to test rain_alert and rain_cleared transitions.

Expected Result	Blynk events trigger email notifications; no duplicates during continuous rain due to lastRainState flag.
Actual Result	Email notifications were received promptly. The lastRainState flag successfully blocked duplicate calls.
Conclusion	Successful; transition-based firing logic functions exactly as designed.

Table 16: Blynk Event Notification Delivery

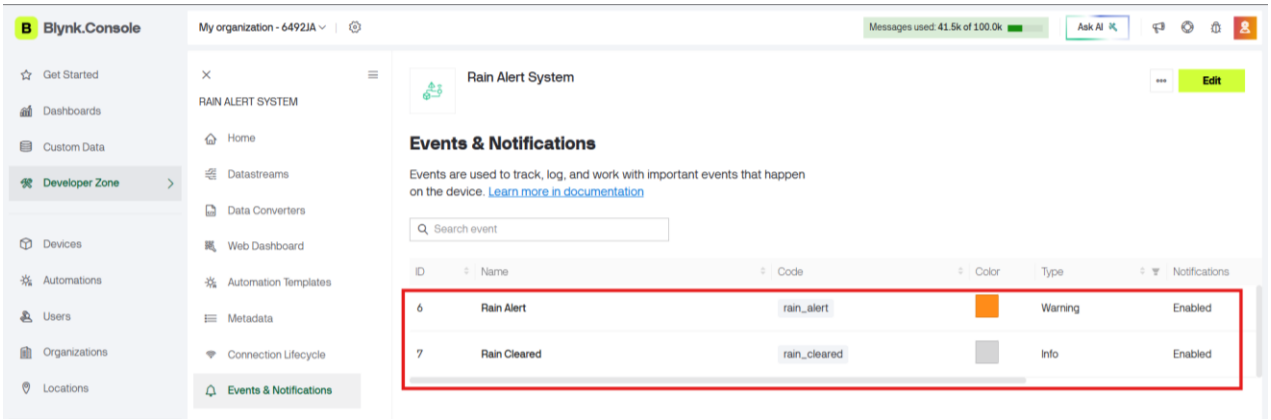

Figure 22: Rain alert/ Rain Clear Event and Notification Creation

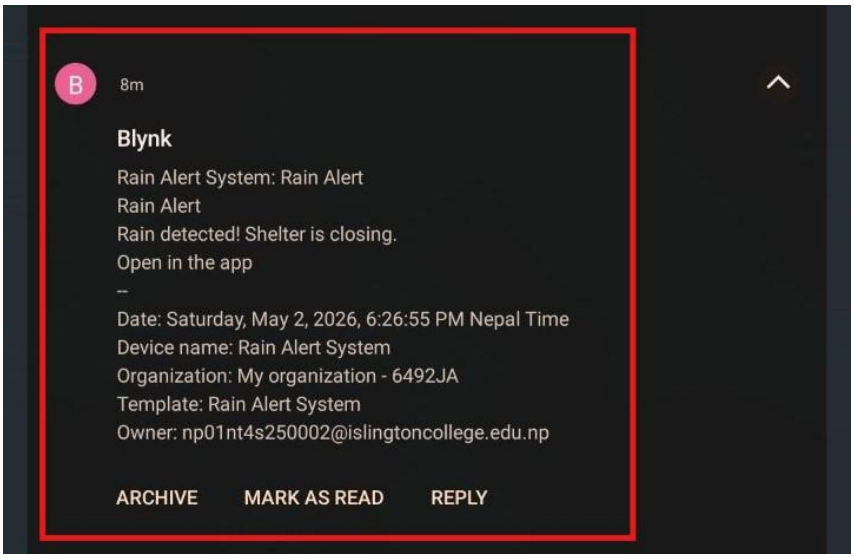

Figure 23: Blynk Event Notification Delivery

5.2.8 Test 8: Complete End-to-End System Integration

Field	Details
Test No.	Test 8
Objective	Complete assembled system performs the full automation cycle reliably.

Activity	Full prototype tested with water application and removals over 5 cycles.
Expected Result	Seamless cycle of detection, actuation, alerts, and IoT notifications.
Actual Result	All cycles executed without failure. Physical mechanism sheltered garments perfectly.
Conclusion	Successful; Complete automation pipeline verified.

Table 17: Complete End-to-End System Integration

6 Advantages and Disadvantages

The IoT-based system provides significant domestic automation benefits while being subject to specific design constraints for wider use (Gorakh, 2024).

6.1 Advantages

- **Complete Automation:** The system operates autonomously, requiring zero human intervention to protect laundry once moisture is detected (Berahim, 2022).
- **Dual-Layer Alert System:** Provides immediate local feedback through a 1 kHz buzzer and remote updates via email notifications, ensuring the user is informed regardless of their proximity to the device.
- **Resource Efficiency:** By preventing clothes from getting wet, the system eliminates the need for repeated washing cycles, saving water, detergent, and time (Wang, 2023).
- **Cost-Effective Design:** Utilizes affordable components like the Arduino Uno, ESP32, and SG90 servo, making it an accessible solution for homeowners (Barral, 2022).
- **Low Complexity:** The circuit design is streamlined with minimal components, which increases overall system reliability and simplifies maintenance.
- **Regional Relevance:** Highly practical for climates with unpredictable weather patterns, such as Nepal, where sudden rainfall is common (Hashim, 2023).

6.2 Disadvantages

- **Sensor Sensitivity:** The rain sensor may occasionally trigger false positives due to heavy dew, dust accumulation, or insects landing on the sensing plate (Setiawan, 2023).
- **Mechanical Load Limits:** The SG90 servo motor has limited torque, making the current prototype suitable only for lightweight garments or small-scale drying racks (Putra, 2023).
- **Platform Limitations:** While the Blynk IoT dashboard offers robust real-time monitoring, advanced features such as long-term historical data logs often require a premium subscription (Medias, 2019).
- **Installation Requirements:** Outdoor setup requires clear sky access for the sensor while shielding microcontrollers from the elements.
- **Scalability Constraints:** The current hardware supports one clothesline; scaling for heavier or multiple lines requires high-torque motors and more power (Zulkipli, 2023).

7 Future Works

The current prototype serves as a successful proof-of-concept for automated laundry protection (Janrao, 2024). However, several enhancements could further improve its reliability, power efficiency, and integration within a smart home ecosystem:

- **High-Torque Actuators:** Transitioning to more powerful motors, such as the DS3225 servo or a DC gear motor paired with an L298N driver, would enable the system to handle the significant weight of heavy, wet laundry (Latif, 2021).
- **Capacitive Sensing Technology:** Replacing the current resistive rain sensor with a capacitive model would improve accuracy by reducing false triggers caused by dew, condensation, or insects (Amale, 2019).
- **Advanced Cloud Logging:** Integrating the ESP32 with platforms like ThingSpeak or AWS IoT Core would facilitate long-term data logging, allowing for historical weather analysis and precipitation patterns (Ahire, 2022).
- **Redundant Notification Channels:** While the system currently utilizes Blynk email notifications, future iterations could integrate Telegram bots or SMS gateways as fall-back channels to ensure the user is notified even during app connectivity issues(Kakani, 2024).
- **Solar Power Autonomy:** Incorporating solar panels with lithium battery backup would make the device fully self-sufficient and suitable for remote outdoor installations without accessible power outlets (Zulkipli, 2023).
- **Smart Home Ecosystems:** Leveraging MQTT to connect with platforms like Google Home or Home Assistant would allow for voice commands and coordinated actions with other smart devices (Sahu, 2024).
- **Multi-Node Coordination:** A single central controller could be used to manage multiple sensor nodes, protecting several different clothesline sections across a larger property (Khoa, 2019).
- **Rain Intensity Logic:** Future firmware could be optimized to categorize rainfall as light, moderate, or heavy based on analog sensor values, adjusting the shelter's response speed or notification priority accordingly (Setiawan, 2023).

8 Conclusion

The project successfully delivered a fully functional IoT-based Automatic Rain Detection and Clothes Protection System designed to address laundry vulnerability during sudden rainfall. The hardware architecture integrates a rain sensor, Arduino Uno, SG90 servo motor, and a piezo buzzer into a unified pipeline, with an ESP32 module serving as the Wi-Fi gateway (Banzi, 2014). The software framework utilizes two specialized firmware programs: the Arduino manages real-time environmental sensing and motor control, while the ESP32 handles cloud connectivity and email event delivery via the Blynk platform (Medias, 2019).

A key technical achievement was the implementation of a dual-threshold detection method that monitors both digital and analog signals to ensure high reliability (Amale, 2019). This approach successfully identified light rainfall that single-output sensors might miss, triggering the servo motor to rotate 90° to shelter the garments. Concurrently, the system provides immediate local feedback through a 1.5-second buzzer alert and transmits real-time data to the Blynk dashboard, including analog values and binary rain status (Setiawan, 2023).

Testing and evaluation confirmed the stability of the system, with email notifications being received within 5 to 15 seconds of a rain event. The UART communication between the Arduino and ESP32 remained robust across multiple test cycles, and the state-transition logic effectively prevented duplicate notification alerts (Foltýnek, 2019). While the current prototype is a success, the project's extensible design provides a clear roadmap for future upgrades, such as high-torque motors and solar power integration, to move from a proof-of-concept toward a professional-grade domestic product (Wang, 2023).

9 References

Ahire, D., 2022. "IoT Based Real-Time Monitoring of Meteorological Data: A Review", s.l.: SSRN Electronic Journal (Review Report), Elsevier.

Amale, O., 2019. "IoT Based Rainfall Monitoring System Using WSN Enabled Architecture". p. pp. 1–5.

Ayaz, S., 2023. "Recent Trends in Cloud Computing and IoT Platforms for IT Management and Development: A Review". p. pp. 98–105.

Banzi, M., 2014. Getting Started with Arduino: The Open Source Electronics Prototyping Platform, 3rd edn.. s.l.:Maker Media / O'Reilly, Sebastopol, CA.

Barral, V., 2022. "Fine Time Measurement for the Internet of Things: A Practical Approach Using ESP32". p. pp. 4386–4397.

Barrett, S., 2013. Arduino Microcontroller Processing for Everyone!, 3rd edn.. s.l.:Morgan & Claypool / Springer, San Rafael, CA.

Berahim, M., 2022. "Automatic Clothes Retriever (ACR)". p. pp. 1241–1248.

Boxall, J., 2013. Arduino Workshop: A Hands-On Introduction with 65 Projects. s.l.:s.n.

Buyya, R., 2016. Internet of Things: Principles and Paradigms. s.l.:Elsevier / Morgan Kaufmann, Amsterdam.

Foltýnek, P., 2019. "Measurement and Data Processing from Internet of Things Modules by Dual-Core Application Using ESP32 Board". p. pp. 1002–1011.

Gadekar, S., 2021. "Arduino Uno-ATmega328P Microcontroller Based Smart Systems", s.l.: SSRN Electronic Journal (Preprint Report), Elsevier.

Gaur, I., 2024. "IoT-Based Smart Home Automation". p. pp. 375–388.

Gorakh, J., 2024. "Rain Detection and Clothes Protection System Using Arduino UNO and Servo Motor". p. pp. 1–7.

Hashim, S., 2023. "Rain Monitoring and Prediction System Using IoT". p. pp. 1–9.

Janrao, V., 2024. "Rain Detection Based Automatic Clothes Collector". p. pp. 1–8.

Kakani, D., 2024. "IoT-Based Smart Home Automation Using ESP32 and Blynk with Enhanced Reliability". p. pp. 1–12.

Kellmereit, D., 2013. The Silent Intelligence: The Internet of Things. s.l.:DND Ventures LLC, San Francisco.

Khoa, T., 2019. "Smart Agriculture Using IoT Multi-Sensors: A Novel Watering Management System". p. pp. 1–18.

Latif, M., 2021. "Design and Development of Smart Automated Clothesline (SAC) Prototype System", s.l.: Universiti Teknikal Malaysia Melaka (UTeM) Technical Report, Malaysia.
Maier, A., 2017. "Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things". p. 143–148.
McEwen, A., 2013. Designing the Internet of Things. s.l.:Wiley, Chichester, UK.
Medias, E., 2019. "Internet of Things (IoT): BLYNK Framework for Smart Home". p. 401–412.
Monk, S., 2012. Programming Arduino: Getting Started with Sketches. s.l.:McGraw-Hill, New York.
Monk, S., 2014. Sensors — A Hands-On Primer for Monitoring the Real World with Arduino and Raspberry Pi. s.l.:Maker Media / O'Reilly, Sebastopol, CA.
Paudel, R., 2020. Lab Manual on Internet of Things, s.l.: NITTTR Chandigarh, India.
Pramudita, R., 2024. "Design of an Automatic Clothes Drying System Based on Internet of Things (IoT) Technology". p. pp. 1–10.
Putra, R., 2023. "Prototype Design of Automatic Clothesline Using Raindrop Sensor and Light Dependent Resistor (LDR) Based on Arduino". p. pp. 1–9.
Sahu, K., 2024. "IoT Based Voice Controlled Home Automation System Using Google Assistant". p. pp. 27–37.
Schwartz, M., 2016. Internet of Things with ESP8266. s.l.:Packt Publishing, Birmingham, UK.
Setiawan, A., 2023. "Design of IoT-Based Automatic Rain-Gauge Radar System for Rainfall Intensity Monitoring". p. pp. 215–230.
Stolojescu-Crisan, C., 2021. "An IoT-Based Smart Home Automation System". p. pp. 1–23.
Wang, K., 2023. "Research on IoT-Based Smart Clothes Drying Rack". p. pp. 1–6.
Zulkipli, M., 2023. "Development of IoT Based Clothesline Using Microcontroller". p. pp. 12–20.

10 Appendix

10.1 Appendix A: Source Code of Arduino (Rain Shelter System)

```
//  Arduino Rain Shelter System
//  Wiring:
//    Rain Sensor VCC  → Arduino 5V
//    Rain Sensor GND  → Arduino GND
//    Rain Sensor DO   → Arduino D2
//    Rain Sensor AO   → Arduino A0
//    Servo Red (5V)   → Arduino 5V
//    Servo Black(GND) → Arduino GND
//    Servo Orange(SIG)→ Arduino D3 (PWM)
//    Buzzer +         → Arduino D9
//    Buzzer -         → Arduino GND
//    Arduino D11 (TX) → ESP32 GPIO16 (RX2)
//    Arduino D10 (RX) → ESP32 GPIO17 (TX2)
//    Arduino GND      → ESP32 GND

#include <Servo.h>
#include <SoftwareSerial.h>
// Pin Definitions
const int RAIN_DO    = 2;
const int RAIN_AO    = A0;
const int SERVO_PIN  = 3;
const int BUZZER_PIN = 9;

// Software Serial for ESP32 Communication
SoftwareSerial espSerial(10, 11);
Servo shelterServo;

// Configuration Constants
const int RAIN_THRESHOLD  = 500;
const unsigned long SIGNAL_INTERVAL = 5000UL;
```

```
// State Variables
bool rainActive     = false;
unsigned long lastSignalTime = 0;

// Function: moveShelter
void moveShelter(int angle) {
  shelterServo.attach(SERVO_PIN);
  shelterServo.write(angle);
  delay(1000);
  shelterServo.detach();
}
// Setup: Runs once when Arduino powers on
void setup() {
  Serial.begin(9600);
  espSerial.begin(9600);
  pinMode(RAIN_DO, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  moveShelter(0);
  Serial.println("System ready. Shelter open.");
}

// Loop: Runs repeatedly after setup
void loop() {
// Step 1: Read Rain Sensor
  int analogSum = 0;
  for (int i = 0; i < 5; i++) {
    analogSum += analogRead(RAIN_AO);
    delay(10);
  }
  int analogVal   = analogSum / 5;
  bool digitalVal = (digitalRead(RAIN_DO) == LOW);
  bool isRaining  = (digitalVal || analogVal < RAIN_THRESHOLD);
```

```
// Step 2: Send Live Analog Value to ESP32
 espSerial.println(analogVal);
// Step 3: Rain Detected
  if (isRaining) {
    if (!rainActive) {
      rainActive = true;
      moveShelter(90);
      tone(BUZZER_PIN, 1000, 1500);
      Serial.println("Rain detected! Shelter closing.");
      espSerial.println("RAIN");
      lastSignalTime = millis();
      Serial.println("SENT: RAIN");
    }
    if (millis() - lastSignalTime >= SIGNAL_INTERVAL) {
      espSerial.println("RAIN");
      lastSignalTime = millis();
      Serial.println("SENT: RAIN repeat");
    }
// Step 4: Rain Stopped
  } else {
    if (rainActive) {
      rainActive = false;
      moveShelter(0);
      noTone(BUZZER_PIN);
      digitalWrite(BUZZER_PIN, LOW);
      espSerial.println("CLEAR");
      Serial.println("Rain stopped. Shelter opening.");
      Serial.println("SENT: CLEAR");
    }
  }
  delay(500);
}
```

10.2 Appendix B: Source Code of ESP32 (Blynk Rain Alert System)

```
// Blynk App Configuratio
#define BLYNK_TEMPLATE_ID   "TMPL3gEIxjBy_"
#define BLYNK_TEMPLATE_NAME "Rain Alert System"
#define BLYNK_AUTH_TOKEN    "zJNUrg1MtsZc4pkx0c8igJYYZVyo7nDz"

#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

// WiFi Credentials
char ssid[] = "test";
char pass[] = "test1234";

// Serial2 Pin Definitions
#define RXD2 16
#define TXD2 17

// Blynk Virtual Pin Definitions
#define VPIN_RAIN_STATUS  V0
#define VPIN_ANALOG_VALUE V1

// State Variable
bool lastRainState = false;

// Blynk Timer
BlynkTimer timer;

// Function: readArduinoData
void readArduinoData() {
  while (Serial2.available()) {
    String line = Serial2.readStringUntil('\n');
```

```
  line.trim();
  if (line.length() == 0) continue;

  Serial.print("[Arduino] >> ");
  Serial.println(line);

  if (line == "RAIN") {
    // Rain Detected
    Blynk.virtualWrite(VPIN_RAIN_STATUS, 1);
    if (!lastRainState) {
      // Fire a Blynk notification event only once when rain first starts
      Blynk.logEvent("rain_alert", "Rain detected! Shelter is closing.");
      Serial.println("[Blynk] Event fired: rain_alert");
      lastRainState = true;
    }
  }
  else if (line == "CLEAR") {
    // Rain Stopped
    Blynk.virtualWrite(VPIN_RAIN_STATUS, 0);
    if (lastRainState) {
      // Fire a Blynk notification event only once when rain stops
      Blynk.logEvent("rain_cleared", "Rain stopped! Shelter is opening.");
      Serial.println("[Blynk] Event fired: rain_cleared");
      lastRainState = false;
    }
  }
  else {
    // Analog Sensor Value
    int val = line.toInt();
    Blynk.virtualWrite(VPIN_ANALOG_VALUE, val);
    Serial.print("[Blynk] Analog sent: ");
    Serial.println(val);
  }
```

```
  }
}
// Setup: Runs once when ESP32 powers on
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  Serial.println("Starting ESP32 Rain Alert System...");
  Serial.println("Connecting to WiFi and Blynk...");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Connected to Blynk!");
  timer.setInterval(300L, readArduinoData);
}
// Loop: Runs repeatedly after setup
void loop() {
  Blynk.run();
  timer.run();
}
```